Spring Circular Dependency, Bean Scope & Bean Initialization — Interview Revision Notes

Topics Covered in This PDF

- Spring DI, IoC and Beans recap
- `@Configuration` and its relationship with `@Component`
- Dependency-based bean creation order
- Circular dependency and circular dependency in normal Java
- Constructor, setter and field injection in circular dependencies
- Spring early references
- Spring Boot circular-reference behavior
- Circular dependency as a design smell and refactoring
- Bean scopes: singleton, prototype, request, session, application, websocket
- Singleton scope and bean-definition meaning
- Prototype scope and prototype inside singleton
- Eager and lazy initialization
- `@Lazy` at bean level and injection point
- Lazy dependency proxies
- Spring Boot global lazy initialization and `@Lazy(false)`
- Eager/lazy dependency combinations
- Using `@Lazy` to break constructor circular dependency
- Bean lifecycle and initialization/destruction callbacks
- Interview questions, quick revision and recall flows

---

1. Spring DI, IoC and Bean — Quick Foundation

Dependency Injection (DI)

Dependency Injection means Spring provides a required dependency to a class instead of the class creating it manually.

```
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Inversion of Control (IoC)

IoC means control of object creation and dependency management is given to the Spring container.

```
Without Spring
Application code → creates objects → wires objects → manages them

With Spring
Spring IoC Container → creates objects → wires objects → manages them
```

Spring Bean

A Spring Bean is an object created, managed and maintained by the Spring IoC container.

Beans can come from:

```
@Component
@Service
@Repository
@Controller
@Bean
```

---

2. `@Configuration` Is Also a Spring Component

Example:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Important:

> `@Configuration` itself is built on top of `@Component`.

Conceptually:

```
@Configuration
      ↓
@Component
      ↓
Spring can detect and manage the class
```

---

3. Bean Creation Order

Spring does not create beans randomly. It creates them according to their dependencies.

Example:

```
OrderController
      ↓
OrderService
      ↓
PaymentService
      ↓
PaymentGateway
```

Therefore the dependency creation order is conceptually:

```
PaymentGateway
      ↓
PaymentService
      ↓
OrderService
      ↓
OrderController
```

> Spring generally creates the dependency before the bean that depends on it.

This becomes important when the dependency graph contains a cycle.

---

4. Circular Dependency

Circular dependency means two or more classes/beans depend on each other directly or indirectly.

```
OrderService → PaymentService
       ↑               |
       └───────────────┘
```

Example:

```
@Service
public class OrderService {
    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {
    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Spring reaches:

```
Create OrderService
        ↓
Need PaymentService
        ↓
Create PaymentService
        ↓
Need OrderService
        ↓
Create OrderService
        ↓
...
```

There is no clear starting point.

---

5. Circular Dependency in Normal Java

Circular dependency is not only a Spring problem.

```
public class A {
    private B b;

    public A() {
        this.b = new B();
    }
}
```

```
public class B {
    private A a;

    public B() {
        this.a = new A();
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        A a = new A();
    }
}
```

Flow:

```
new A()
  ↓
A constructor
  ↓
new B()
  ↓
B constructor
  ↓
new A()
  ↓
A constructor
  ↓
new B()
  ↓
...
```

Eventually:

```
StackOverflowError
```

because constructor calls keep stacking.

> Circular dependency is fundamentally a design problem; Spring exposes it during bean creation.

6. Constructor-Based Circular Dependency

Constructor injection requires all constructor arguments before the object can be created.

```
public OrderService(PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

If `PaymentService` also needs `OrderService`:

```
Create OrderService
        ↓
Needs PaymentService
        ↓
Create PaymentService
        ↓
Needs OrderService
        ↓
...
```

Therefore constructor-based circular dependency normally fails.

A common exception is:

```
BeanCurrentlyInCreationException
```

The application fails during startup.

7. Why Constructor Injection Is Still Preferred

Constructor injection is generally preferred because:

- Dependencies are explicit.
- Required dependencies are mandatory.
- Dependencies can be `final`.

- The object is complete after construction.
- Testing is easier without Spring.

Example:

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Important interview statement:

> Constructor injection is not the problem. The circular design is the problem.

---

8. Setter and Field Injection in Circular Dependency

Setter injection separates object creation from dependency injection.

```
@Service
public class OrderService {
    private PaymentService paymentService;

    @Autowired
    public void setPaymentService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {
    private OrderService orderService;

    @Autowired
    public void setOrderService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Conceptually Spring can:

```
1. Create OrderService
2. Create PaymentService
3. Inject PaymentService into OrderService
4. Inject OrderService into PaymentService
```

Field injection follows the same basic idea:

```
@Service
public class OrderService {
    @Autowired
    private PaymentService paymentService;
}
```

```
@Service
public class PaymentService {
    @Autowired
    private OrderService orderService;
}
```

Because objects can be created before their dependencies are injected, Spring may resolve some setter/field circular dependencies.

> This does not make circular dependency good design.

---

9. Spring Early Reference

For a normal dependency:

```
A needs B
   ↓
Create B
   ↓
Inject B's dependencies
   ↓
Initialize B
   ↓
B ready
   ↓
Create A
   ↓
Inject B into A
```

In some circular cases Spring can expose an early reference.

Conceptually:

```
A a = new A();  // exists, not fully initialized
B b = new B();

b.setA(a);      // B receives early A
a.setB(b);      // A receives B
```

An early reference means Spring exposes a reference to a bean before that bean has completed full initialization so another bean can temporarily use it during circular-dependency resolution.

---

10. Spring Boot Circular References

Spring Framework has mechanisms for some circular references, but Spring Boot discourages them.

From Spring Boot 2.6 onward, circular references are disabled by default.

```
spring.main.allow-circular-references=false
```

Default:

```
false
```

It can technically be enabled:

```
spring.main.allow-circular-references=true
```

But enabling it should not be considered the proper architectural solution.

---

11. Circular Dependency as a Design Smell

If:

```
OrderService ↔ PaymentService
```

ask:

```
Why does A need B?
Why does B need A?
Are responsibilities mixed?
Can common logic move elsewhere?
Can one dependency direction be removed?
```

Better designs may look like:

```
OrderService → PaymentService
```

or:

```
OrderService → PaymentProcessor
PaymentService → PaymentProcessor
```

or:

```
OrderService → PaymentService
PaymentService → PaymentStatusRepository
```

Best ways to solve it

Problem	Better design
Two services call each other	Move shared logic to a third service
One service has too many responsibilities	Split responsibilities
Both depend on internal behavior	Introduce interface/event-based communication
Orchestration and business logic are mixed	Create coordinator/orchestrator

> Best long-term solution: refactor and remove the cycle.

---

12. Bean Scope

Bean scope determines:

1. How many objects Spring creates for a bean definition.
2. How long those objects live.

Important scopes:

```
singleton
prototype
request
session
application
websocket
```

The two main core scopes are:

```
singleton
prototype
```

---

13. Singleton Scope

Singleton is the default Spring bean scope.

Meaning:

> For one bean definition inside one Spring container, Spring creates one shared object and returns the same object when requested.

```
@Component
public class PaymentService {
}
```

is singleton by default.

Explicit version:

```
@Component
@Scope("singleton")
public class PaymentService {
}
```

Usually `@Scope("singleton")` is unnecessary.

---

14. Singleton Example

```
ApplicationContext context =
        new AnnotationConfigApplicationContext(AppConfig.class);

PaymentService p1 =
        context.getBean(PaymentService.class);

PaymentService p2 =
        context.getBean(PaymentService.class);

System.out.println(p1 == p2);
```

Output:

```
true
```

Both references point to the same Spring-managed object.

```
        Spring Container
              |
       PaymentService
              |
         +----+----+
         |         |
        p1        p2
         \         /
          same object
```

---

15. Spring Singleton ≠ JVM Singleton

Spring singleton does **not** mean only one object of that class can ever exist in the JVM.

You can still write:

```
PaymentService p = new PaymentService();
```

That object is simply not managed by Spring.

Spring singleton means:

> One shared object for that bean definition inside that Spring container.

---

16. Singleton Is Per Bean Definition, Not Per Class

Example:

```
@Bean
public User createUser() {
    return new User();
}

@Bean
public User createUser2() {
    return new User();
}
```

There are two bean definitions:

```
createUser
createUser2
```

Both are singleton by default:

```
createUser  → User object #1
createUser2 → User object #2
```

So:

```
2 bean definitions
2 singleton objects
```

Important:

> Spring singleton means one object per bean definition inside the container, not one object per Java class.

---

17. Prototype Scope

Prototype means:

> Spring creates a new object every time that prototype bean is requested from the container.

```
@Component
@Scope("prototype")
public class OrderRequest {

    public OrderRequest() {
        System.out.println("OrderRequest object created");
    }
}
```

```
OrderRequest r1 =
        context.getBean(OrderRequest.class);

OrderRequest r2 =
        context.getBean(OrderRequest.class);

System.out.println(r1 == r2);
```

Output:

```
false
```

Flow:

```
getBean()
   ↓
new object

getBean()
   ↓
another new object
```

The object is still created and configured by Spring.

---

18. Singleton vs Prototype — When to Use

Scope	Common use
Singleton	Stateless services/behavior
Prototype	Stateful objects needing fresh instances

Services are usually singleton:

```
@Service
public class PaymentService {
    public void processPayment() {
    }
}
```

Prototype can be useful for fresh state:

```
@Component
@Scope("prototype")
public class OrderRequest {
}
```

---

19. Prototype Injected into Singleton

Important interview question:

> If a prototype bean is injected into a singleton, does the singleton receive a new prototype every time a method runs?

No.

Suppose:

```
OrderService = singleton
OrderRequest = prototype
```

```
@Service
public class OrderService {

    private final OrderRequest orderRequest;

    public OrderService(OrderRequest orderRequest) {
        this.orderRequest = orderRequest;
    }
}
```

Conceptually:

```
Create singleton OrderService
          ↓
Request prototype OrderRequest
          ↓
Create OrderRequest #1
          ↓
Inject into OrderService
          ↓
OrderService keeps reference #1
```

Later method calls use the same injected reference.

Prototype means:

> New object whenever the prototype bean is requested from the container.

It does not mean:

> New object whenever a singleton method runs.

---

20. Web Scopes

Spring also provides:

Scope	One object for
`request`	One HTTP request
`session`	One user session
`application`	One web application / `ServletContext`
`websocket`	One WebSocket session

These require a web-aware Spring application context.

For Spring MVC/Boot interviews, remember especially:

```
request
session
application
```

---

21. Request, Session and Application Scope

Request

```
HTTP request 1 → Bean #1
HTTP request 2 → Bean #2
```

Useful for data that lives only during one request.

Session

```
User A session → Bean A
User B session → Bean B
```

Useful for user-specific session data.

Application

One bean for the web application, tied to `ServletContext`.

```
Web Application
      ↓
Application-scoped bean
```

---

22. Bean Initialization — Eager vs Lazy

Spring must decide when to create a bean.

Type	Meaning
Eager	Create during application startup
Lazy	Create only when actually needed

Default:

```
Singleton → eager
Prototype → created when requested
```

`@Lazy` can delay creation.

---

23. Eager Initialization

Eager initialization means the bean is created when the application context starts.

```
@Component
public class PaymentService {

    public PaymentService() {
        System.out.println("PaymentService object created");
    }
}
```

Flow:

```
Application starts
      ↓
Spring creates singleton
      ↓
Application continues
```

Why eager is the default

It provides fail-fast behavior.

```
Startup
   ↓
Create beans
   ↓
Missing dependency/configuration
   ↓
Fail immediately
```

Benefits:

```
Fast failure
Early error detection
Predictable startup validation
```

---

24. Lazy Initialization

Lazy initialization means:

> Do not create the bean during startup. Create it only when needed.

```
@Component
@Lazy
public class ReportService {

    public ReportService() {
        System.out.println("ReportService object created");
    }

    public void generateReport() {
        System.out.println("Report generated");
    }
}
```

At startup, the object is not created.

Later:

```
ReportService reportService =
        context.getBean(ReportService.class);

reportService.generateReport();
```

Output:

```
Application started
ReportService object created
Report generated
```

Spring already knows the BeanDefinition; the actual object is delayed.

---

25. BeanDefinition During Lazy Initialization

A BeanDefinition can contain information such as:

```
Bean name
Class
Scope
Dependencies
Lifecycle details
Lazy/eager behavior
```

Lazy flow:

```
Application startup
       ↓
BeanDefinition exists
       ↓
No actual object yet
       ↓
Bean requested
       ↓
Create object
       ↓
Initialize
       ↓
Return bean
```

---

26. Eager Bean Depends on Lazy Bean

Suppose:

```
UserService = eager
ReportService = lazy
```

and:

```
UserService → ReportService
```

If `UserService` directly requires `ReportService`, Spring may create `ReportService` during startup to satisfy the eager bean.

```
Startup
  ↓
Create UserService
  ↓
Needs ReportService
  ↓
Create ReportService
  ↓
Inject it
  ↓
UserService ready
```

So `@Lazy` on a bean does not guarantee that it will never be created at startup.

---

27. `@Lazy` at Injection Point and Proxy

Instead of making the whole dependency bean lazy, use:

```
@Component
public class UserService {

    private final EmailService emailService;

    public UserService(@Lazy EmailService emailService) {
        this.emailService = emailService;
    }

    public void registerUser() {
        System.out.println("User registered");
        emailService.sendEmail();
    }
}
```

Spring can inject a proxy.

Conceptually:

```
UserService
     |
     v
   Proxy
     |
     | later when used
     v
Real EmailService
```

When:

```
emailService.sendEmail();
```

is called, Spring resolves/creates the real dependency.

---

28. Bean-Level `@Lazy` vs Injection-Point `@Lazy`

Bean-level

```
@Component
@Lazy
public class ReportService {
}
```

Meaning:

> Delay creation of this bean.

Injection point

```
public UserService(@Lazy EmailService emailService) {
}
```

Meaning:

> Inject a proxy and delay resolving the real dependency until it is needed.

Recall:

```
@Lazy on bean
    → delay bean creation

@Lazy on injection point
    → proxy + delayed dependency resolution
```

---

29. Global Lazy Initialization in Spring Boot

Enable:

```
spring.main.lazy-initialization=true
```

or:

```
spring:
  main:
    lazy-initialization: true
```

This makes bean creation lazy across the application.

Benefit

Can reduce startup time.

Trade-off

Configuration errors can appear later:

```
Application starts
      ↓
Lazy bean not created
      ↓
Feature requested
      ↓
Bean created
      ↓
Error discovered at runtime
```

Default:

```
spring.main.lazy-initialization=false
```

---

30. `@Lazy(false)`

If global lazy initialization is enabled, a particular bean can be made eager:

```
@Component
@Lazy(false)
public class ImportantStartupBean {
}
```

Meaning:

> Create this bean eagerly even though global lazy initialization is enabled.

---

31. Common Eager/Lazy Combinations

Eager → Lazy

```
Eager A → Lazy B
```

B may still be created during startup because A needs it.

Use `@Lazy` at the injection point if the dependency itself should be delayed.

Lazy → Eager

```
Lazy A → Eager B
```

B is normally created at startup.

When A is later created, Spring injects the already-created B.

Lazy → Lazy

```
Lazy A → Lazy B
```

Neither needs to be created at startup.

When A is requested:

```
Create A
  ↓
A needs B
  ↓
Create B
```

With `@Lazy` at the injection point, B may be represented by a proxy until actually used.

---

32. `@Lazy` and Constructor Circular Dependency

Sometimes `@Lazy` can break a constructor circular dependency.

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(@Lazy PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

```
@Service
public class PaymentService {

    private final OrderService orderService;

    public PaymentService(OrderService orderService) {
        this.orderService = orderService;
    }
}
```

Conceptually:

```
Create OrderService
      ↓
Needs PaymentService
      ↓
@Lazy → inject proxy
      ↓
OrderService can finish
      ↓
PaymentService can be created
      ↓
Real PaymentService resolved later
```

Warning

`@Lazy` should be treated as a workaround, not the best design.

Use it when:

- Refactoring is not immediately possible.
- You are working with legacy code.
- You understand the dependency flow.
- A temporary solution is required.

Best long-term solution:

```
Fix design
   ↓
Separate responsibilities
   ↓
Remove cycle
```

---

33. Bean Lifecycle

Bean lifecycle means the complete journey of a bean from discovery to destruction.

```
Bean Definition
      ↓
Object Creation
      ↓
Dependency Injection
      ↓
Initialization
      ↓
Ready to Use
      ↓
Destruction
```

---

34. Lifecycle Step 1 — Bean Definition

Spring discovers a bean through:

```
@Component
@Service
@Repository
@Controller
@Bean
XML
```

Spring stores metadata such as:

```
Bean name
Class name
Scope
Dependencies
Lazy/eager behavior
Lifecycle methods
```

At this stage, the actual object has not necessarily been created.

---

35. Lifecycle Step 2 — Object Creation

Spring creates the actual object using its constructor.

Conceptually:

```
PaymentService paymentService =
        new PaymentService();
```

But application code does not manually create this Spring-managed object; the container does.

```
BeanDefinition
      ↓
Spring creates object
      ↓
Bean instance
```

---

36. Lifecycle Step 3 — Dependency Injection

After object creation, Spring injects required dependencies.

```
@Service
public class OrderService {

    private final PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Flow:

```
Create OrderService
      ↓
Resolve PaymentService
      ↓
Inject PaymentService
```

---

37. Lifecycle Step 4 — Initialization

After dependency injection, Spring runs initialization logic.

Typical tasks:

```
Validate configuration
Open resources
Load required data
Check required fields
Prepare state
```

Common mechanisms:

```
@PostConstruct
InitializingBean
Custom init method
```

Example:

```
@PostConstruct
public void init() {
    System.out.println("Bean initialized");
}
```

---

38. Lifecycle Step 5 — Ready to Use

After initialization:

```
Bean is ready
```

Other beans can call its methods.

```
Definition
   ↓
Creation
   ↓
Injection
   ↓
Initialization
   ↓
Ready
```

---

39. Lifecycle Step 6 — Destruction

When the Spring container shuts down, singleton beans can run destruction callbacks.

Typical cleanup:

```
Close resources
Release connections
Stop background tasks
Clean state
```

Mechanisms:

```
@PreDestroy
DisposableBean
Custom destroy method
```

Example:

```
@PreDestroy
public void cleanup() {
    System.out.println("Cleaning resources");
}
```

---

40. Prototype Destruction

Important:

> Spring creates and injects prototype beans, but does not fully manage their destruction lifecycle in the same way as singleton beans.

Recall:

```
Singleton
→ full Spring lifecycle including destruction callbacks

Prototype
→ created/injected by Spring
→ destruction is not fully managed by Spring
```

---

41. Circular Dependency vs Bean Lifecycle

These are related but different.

Circular Dependency asks:

> Can Spring create the objects and resolve their dependencies?

```
A → B
↑   |
└───┘
```

Bean Lifecycle asks:

> What happens to a bean from discovery until destruction?

```
Definition
    ↓
Creation
    ↓
Injection
    ↓
Initialization
    ↓
Ready
    ↓
Destruction
```

Circular dependency mainly causes trouble during creation/dependency resolution.

---

42. Interview Questions

Q1. What is circular dependency?

Two or more beans depend on each other directly or indirectly, creating a cycle.

Q2. Why does constructor injection fail for circular dependencies?

Constructor arguments must exist before the object can be created. In A → B → A, there is no valid starting point.

Q3. Why can setter injection sometimes resolve it?

Objects can be created first and dependencies injected later.

Q4. Can field injection resolve it?

Some field-based circular dependencies can be resolved because fields are injected after object creation.

Q5. What is an early reference?

A reference exposed before full bean initialization so Spring can resolve some circular dependencies.

Q6. Does Spring Boot allow circular references by default?

No. Spring Boot discourages them and from 2.6 onward disables circular references by default.

Q7. Best way to solve circular dependency?

Refactor the design: separate responsibilities, move shared logic, introduce appropriate interfaces/events, or create an orchestrator.

Q8. What is singleton scope?

One shared object per bean definition inside a Spring container.

Q9. Is Spring singleton the same as Singleton Design Pattern?

No. Spring singleton is container/bean-definition scoped and does not prevent manual object creation.

Q10. Can one class have multiple singleton beans?

Yes. Different bean definitions can produce different singleton objects of the same class.

Q11. What is prototype scope?

A new object is created whenever the prototype bean is requested from the container.

Q12. Does a prototype injected into a singleton create a new object on every method call?

No. The singleton normally keeps the prototype reference injected during its creation.

Q13. What are important web scopes?

`request`, `session`, `application`, `websocket`.

Q14. What is eager initialization?

Bean creation during application startup.

Q15. Why is eager initialization useful?

It provides fail-fast behavior and detects configuration/dependency problems early.

Q16. What is lazy initialization?

Bean creation is delayed until the bean is needed.

Q17. What does `@Lazy` do?

It can delay bean creation or, at an injection point, delay dependency resolution using a proxy.

Q18. `@Lazy` on bean vs injection point?

```
Bean-level @Lazy
→ delay that bean's creation

Injection-point @Lazy
→ inject proxy and delay real dependency resolution
```

Q19. What happens when an eager bean directly needs a lazy bean?

The lazy bean may be created during startup because it is needed to create the eager bean.

Q20. What is global lazy initialization?

```
spring.main.lazy-initialization=true
```

It makes bean creation lazy by default across the application.

Q21. Default global lazy setting?

```
spring.main.lazy-initialization=false
```

Q22. How can one bean remain eager under global lazy initialization?

```
@Lazy(false)
```

Q23. Can `@Lazy` break constructor circular dependency?

Sometimes, by injecting a proxy. It should generally be considered a workaround.

Q24. What is Bean lifecycle?

```
Definition
→ Creation
→ Dependency Injection
→ Initialization
→ Ready
→ Destruction
```

Q25. Common initialization callbacks?

```
@PostConstruct
InitializingBean
Custom init method
```

Q26. Common destruction callbacks?

```
@PreDestroy
DisposableBean
Custom destroy method
```

Q27. Does Spring fully manage prototype destruction?

No. Prototype destruction is not managed in the same complete way as singleton destruction.

QUICK REVISION

1. Circular Dependency

```
A → B
↑   |
└───┘
```

> Beans depend on each other directly or indirectly.

Constructor

```
A needs B
B needs A
   ↓
No starting point
   ↓
Creation fails
```

Common exception:

```
BeanCurrentlyInCreationException
```

Setter / Field

```
Create A
Create B
Inject B → A
Inject A → B
```

May work because creation and injection are separate.

Best solution

```
Refactor
  ↓
Separate responsibilities
  ↓
Remove cycle
```

`@Lazy` can help in some cases but is generally a workaround.

---

2. Bean Scope

```
singleton
prototype
request
session
application
websocket
```

Singleton

```
One object
per bean definition
per Spring container
```

Not one object per Java class in the entire JVM.

Prototype

```
getBean()
   ↓
new object

getBean()
   ↓
new object
```

---

3. Scope Comparison

Scope	Recall
Singleton	One object per bean definition/container
Prototype	New object per container request
Request	One object per HTTP request
Session	One object per user session
Application	One object per web application
WebSocket	One object per WebSocket session

---

4. Eager vs Lazy

Eager

```
Application starts
      ↓
Bean created
```

Default for singleton beans.

Main advantage:

> Fail fast.

Lazy

```
Application starts
      ↓
Bean not created
      ↓
Bean needed
      ↓
Bean created
```

5. `@Lazy`

Bean level

```
@Component
@Lazy
public class ReportService {
}
```

→ Delay bean creation.

Injection point

```
public UserService(@Lazy EmailService emailService) {
}
```

→ Inject proxy; resolve real bean later.

Global Boot

```
spring.main.lazy-initialization=true
```

Default:

```
spring.main.lazy-initialization=false
```

Force eager:

```
@Lazy(false)
```

6. Bean Lifecycle

Memorize:

```
1. Bean Definition
        ↓
2. Object Creation
        ↓
3. Dependency Injection
        ↓
4. Initialization
        ↓
5. Ready to Use
        ↓
6. Destruction
```

Initialization:

```
@PostConstruct
InitializingBean
Custom init method
```

Destruction:

```
@PreDestroy
DisposableBean
Custom destroy method
```

30-SECOND INTERVIEW RECALL

```
Spring creates beans based on dependency order
                ↓
A → B → C = normal dependency chain
                ↓
A → B → A = circular dependency
                ↓
Constructor injection usually fails
                ↓
Setter/field may sometimes work
                ↓
Early reference may help
                ↓
Spring Boot discourages circular references
                ↓
Best fix = refactor design
                ↓
Bean Scope
                ↓
Singleton = one per bean definition/container
                ↓
Prototype = new object per container request
                ↓
Request/session/application/websocket = web scopes
                ↓
Initialization
                ↓
Singleton = eager by default
                ↓
@Lazy = delayed creation
                ↓
@Lazy injection = proxy
                ↓
Global lazy = spring.main.lazy-initialization=true
                ↓
Bean Lifecycle
                ↓
Definition → Creation → Injection
                ↓
Initialization → Ready → Destruction
```

Final Things to Never Forget

1. Circular dependency is usually a **design problem**.
2. Constructor injection normally fails for a circular dependency.
3. Setter/field injection may resolve some circular dependencies because creation and injection are separate.
4. `@Lazy` can sometimes break a constructor cycle using a proxy, but refactoring is preferred.
5. Spring singleton = one object per bean definition inside one container.
6. Prototype = new object whenever the prototype bean is requested from the container.
7. A prototype injected into a singleton does not automatically become new on every method call.
8. Singleton beans are eagerly initialized by default.
9. `@Lazy` delays bean creation; injection-point `@Lazy` can delay dependency resolution with a proxy.
10. Global lazy initialization uses `spring.main.lazy-initialization=true`.
11. Bean lifecycle = Definition → Creation → Injection → Initialization → Ready → Destruction.
12. Prototype destruction is not managed in the same complete way as singleton destruction.